

W10. General-Purpose computing on Graphics Processing Units (GPGPU)

Department of Smart Mobility Engineering

Inwook, Shim

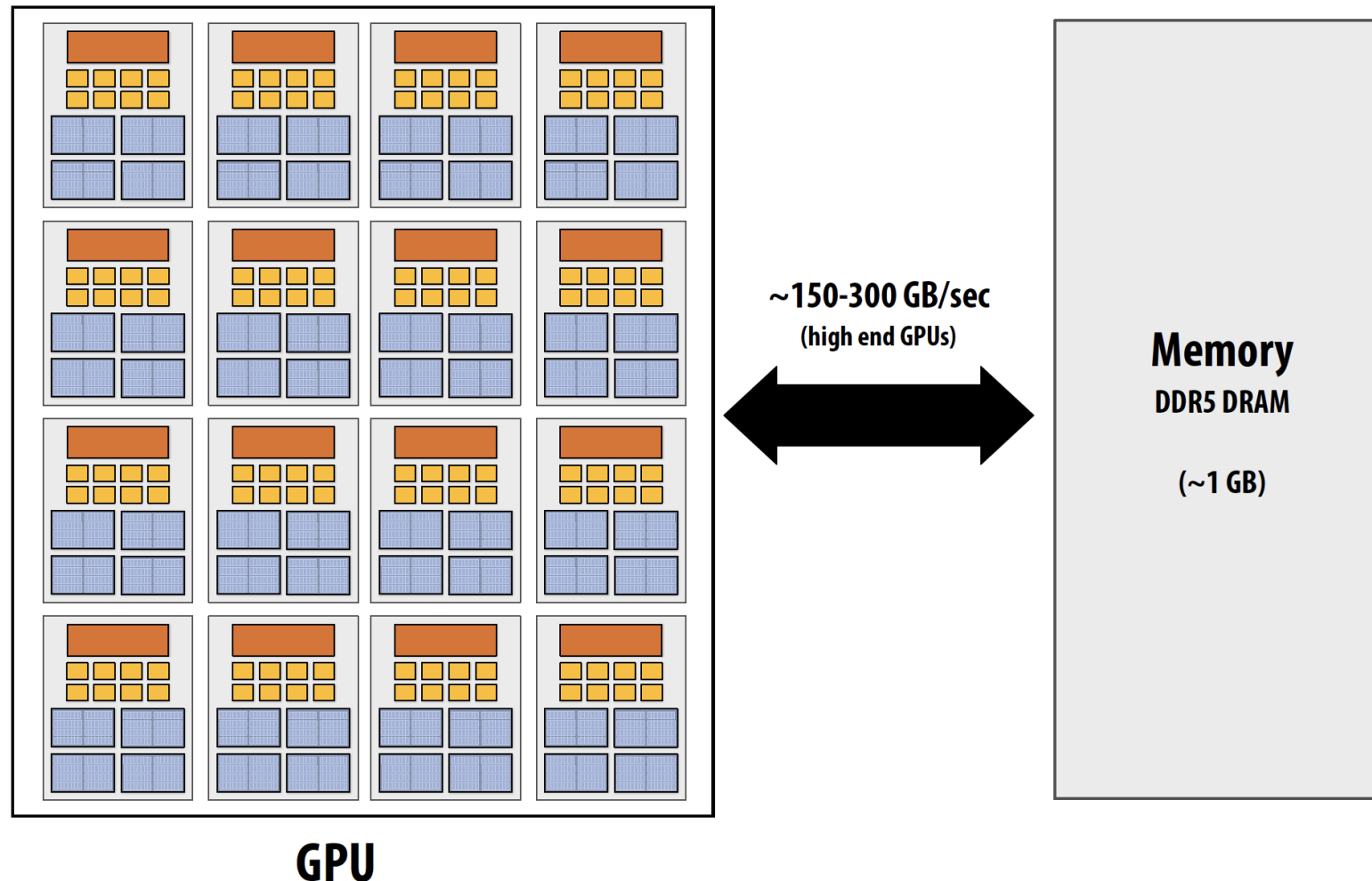


인하대학교
INHA UNIVERSITY

GPU Architecture and CUDA Programming

- History: how graphics processors, originally designed to accelerate 3D games like Quake, evolved into highly parallel compute engines for a broad class of applications
- Programming GPUs using the CUDA language
- A more detailed look at GPU architecture

basic GPU architecture



Multi-core chip

SIMD execution within a single core (many execution units performing the same instruction)

Multi-threaded execution on a single core (multiple threads executed concurrently by a core)

basic GPU architecture

- What GPUs were originally designed to do: 3D rendering

Utah Teapot

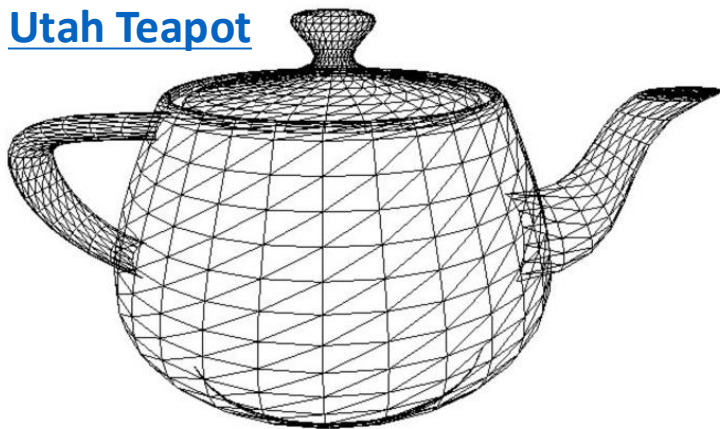


Image credit: Henrik Wann Jensen

Input: description of a scene:

3D surface geometry (e.g., triangle mesh)
surface materials, lights, camera, etc.

Output: image of the scene

Simple definition of rendering task: computing how each triangle in 3D mesh contributes to appearance of each pixel in the image?

GPUs are designed to do

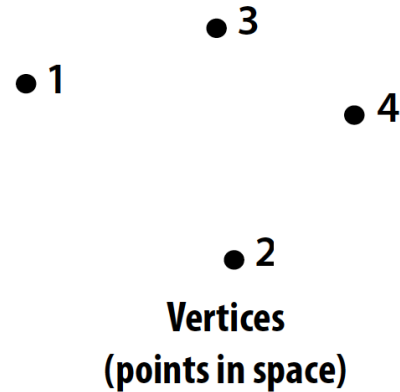
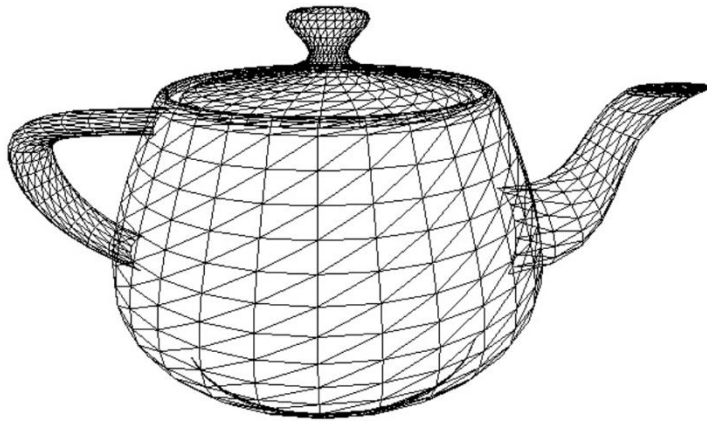


Gran Turismo Sport © 2017 Sony Interactive Entertainment Inc. Developed by Polyphony Digital Inc. All rights reserved. Any depiction or recreation of real-world entities, businesses, or organizations is not intended to be or imply any sponsorship or endorsement of this game by such party or parties. "Gran Turismo" logos are registered trademarks or trademarks of Sony Interactive Entertainment Inc. "Captured on PS5"

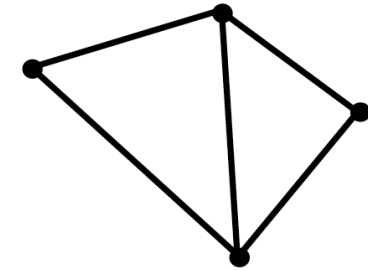
GRAN TURISMO 7
THE REAL DRIVING SIMULATOR

How it works ?

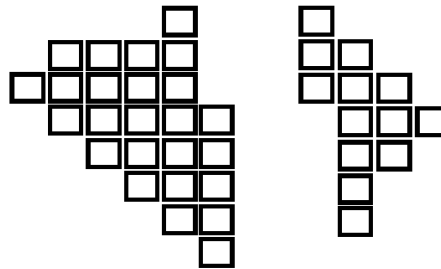
- Step 1: describe the things (key entities) that are manipulated
 - The nouns



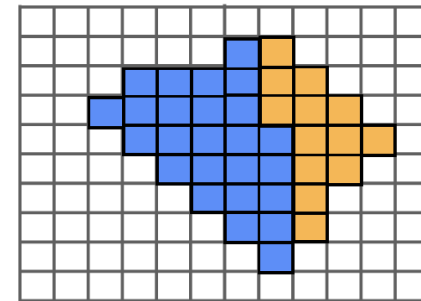
Vertices
(points in space)



Primitives
(e.g., triangles, points, lines)



Fragments



Pixels (in an image)

How it works ?

- Step 1: describe the things (key entities) that are manipulated - The nouns
- Step 2: describe operations the system performs on the entities - The verbs

How it works ?

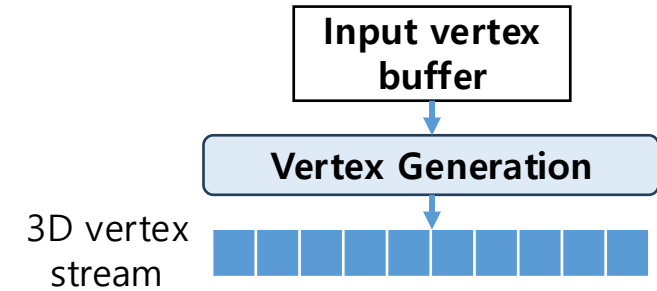
- Rendering a picture
 - Input: a list vertices in 3D space
(and their connectivity into primitives)

Example: every three vertices defines a triangle

```
list_of_positions = {  
    { v0x, v0y, v0z, }  
    { v1x, v1y, v1z, }  
    { v2x, v2y, v2z, }  
    { v3x, v3y, v3z, }  
};
```

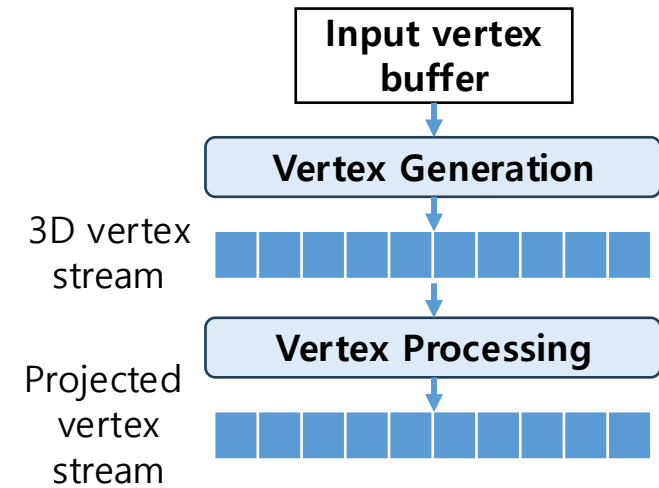
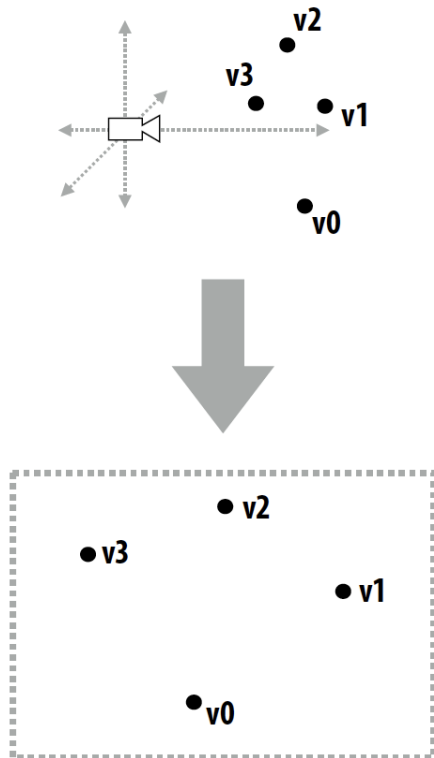
triangle 0 = {v0, v1, v2}
triangle 1 = {v1, v2, v3}

triangle = { {0, 1, 2},
 {1, 2, 3},
 {
 :
 } }



How it works ?

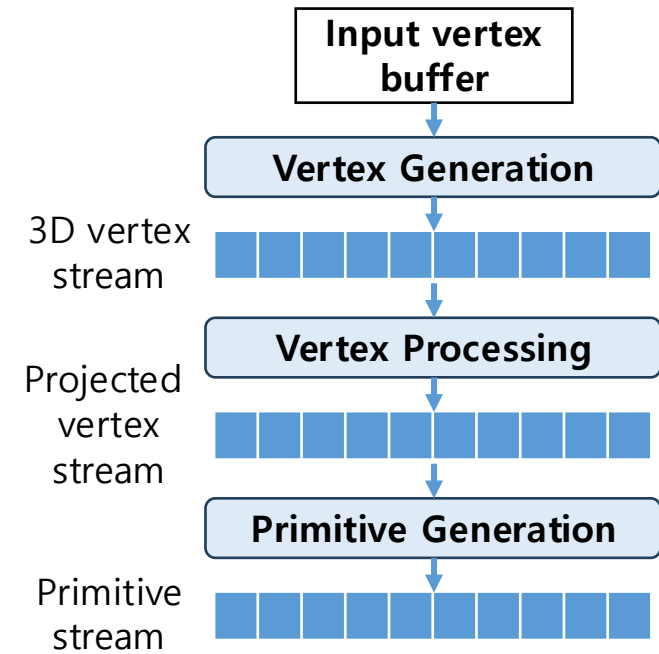
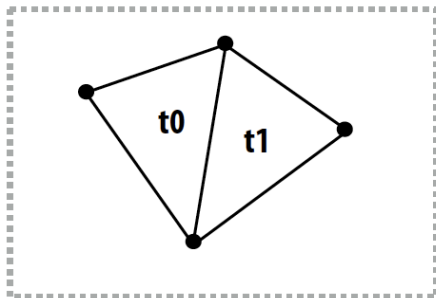
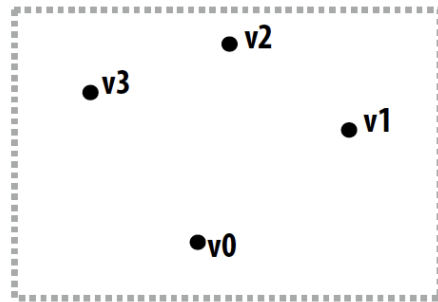
- Rendering a picture
 1. given a scene camera position,
compute where the vertices lie on screen



How it works ?

- Rendering a picture

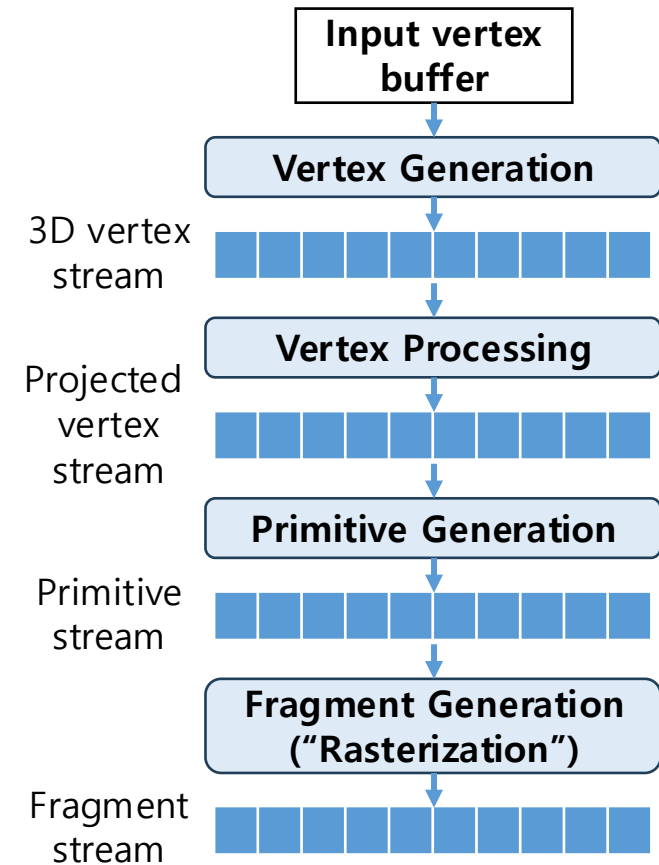
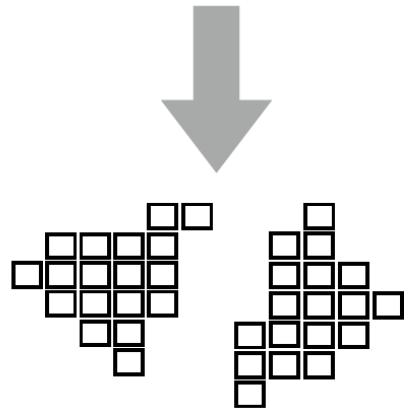
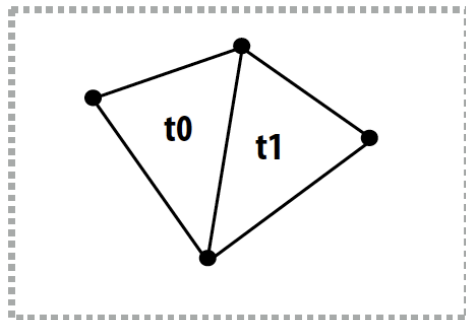
2. group vertices into primitives



How it works ?

- Rendering a picture

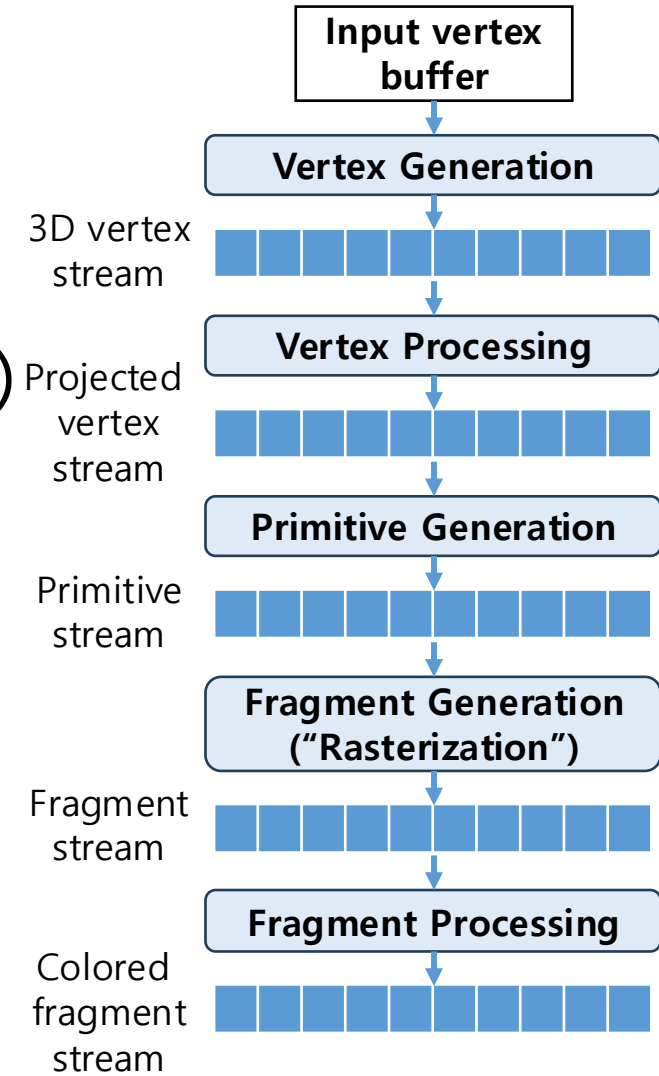
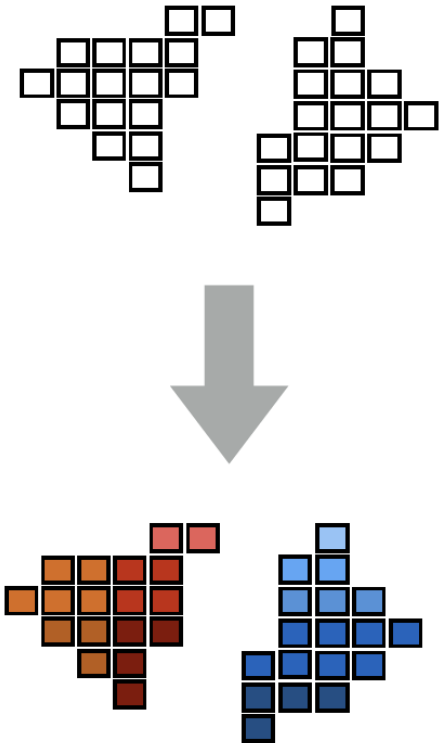
3. generate one fragment for each pixel a primitive overlaps



How it works ?

- Rendering a picture

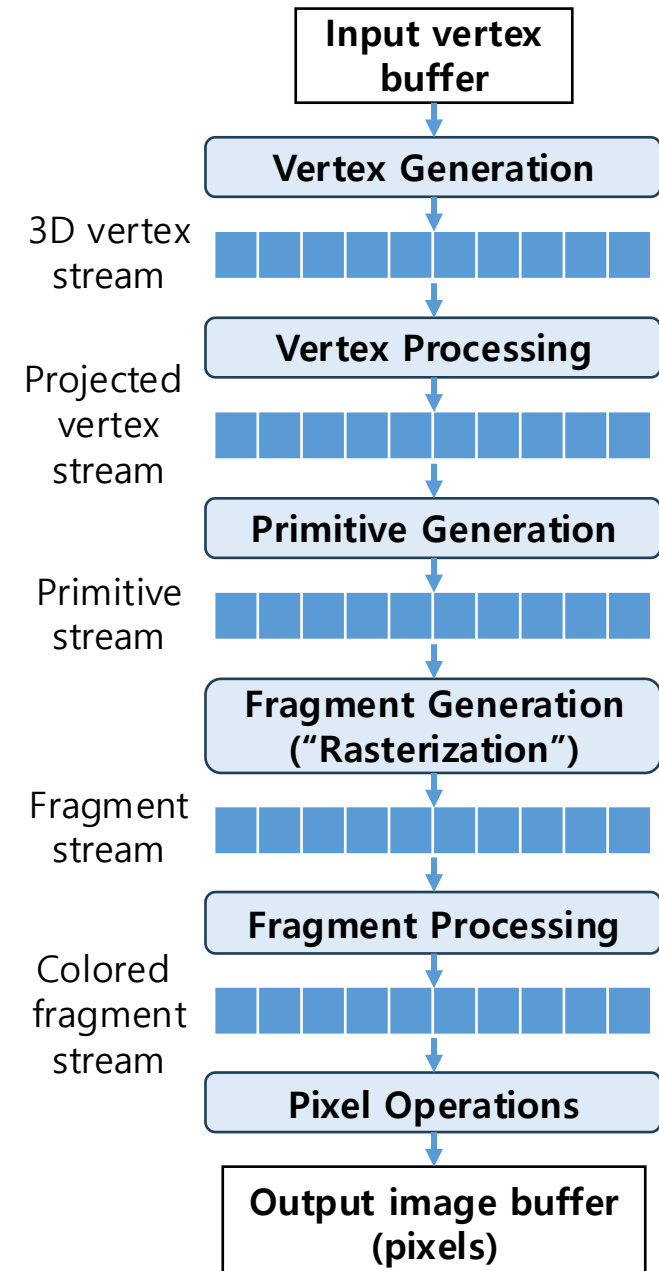
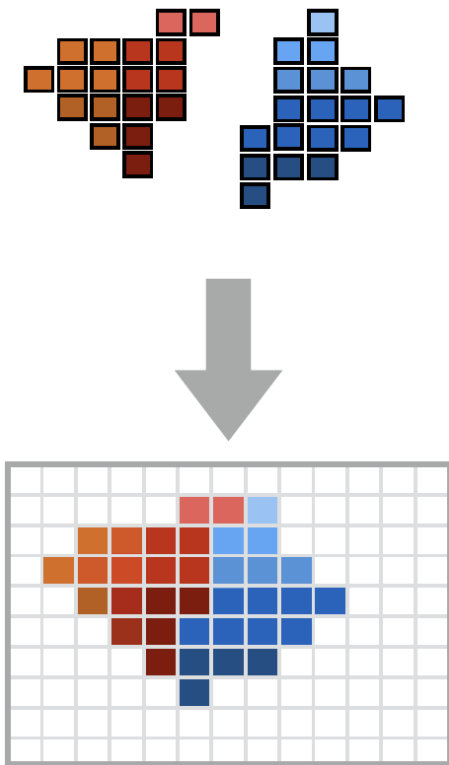
4. compute color of primitive for each fragment
(based on scene lighting and primitive material properties)



How it works ?

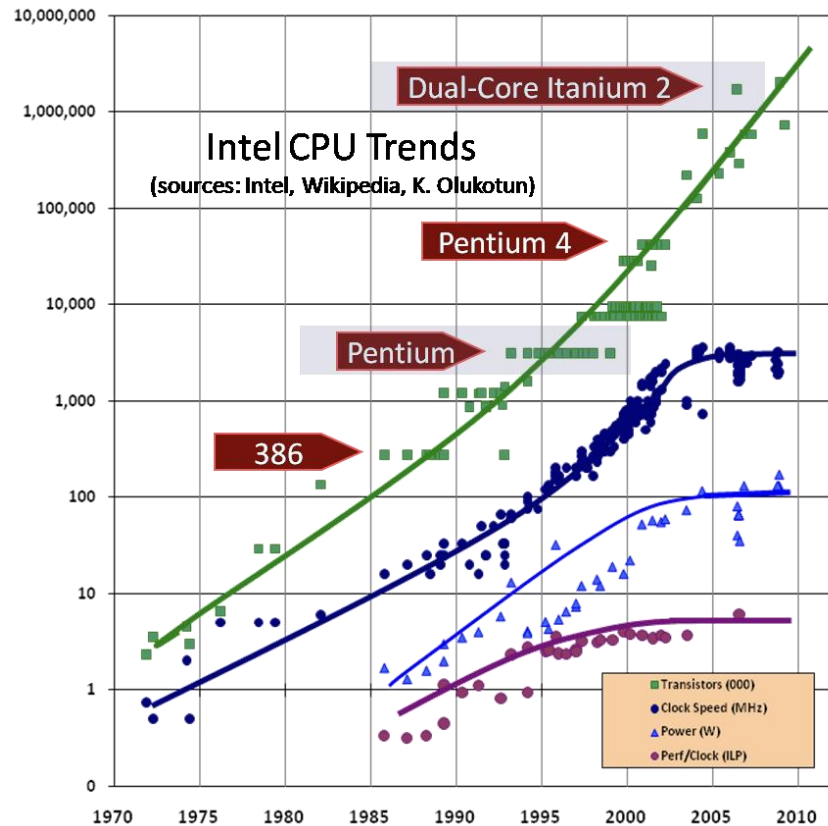
- Rendering a picture

5. put color of the “closest fragment” to the camera in the output image



GPUs in 2001~2003

- These GPUs are very fast processors for performing the same computation on large collections of data (streams of vertices, fragments, and pixels)



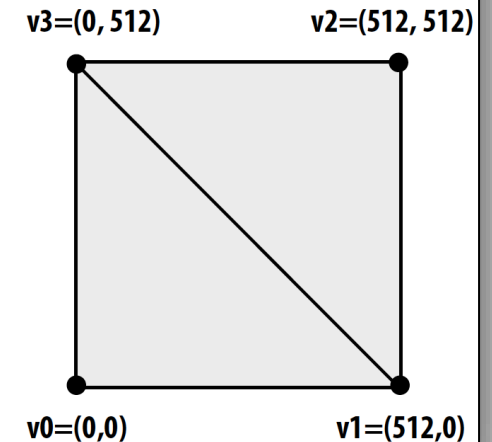
Set OpenGL output image size to be output array size (e.g., 512 x 512)

Render 2 triangles that exactly cover screen

(one shader computation per pixel = one shader computation output image element)

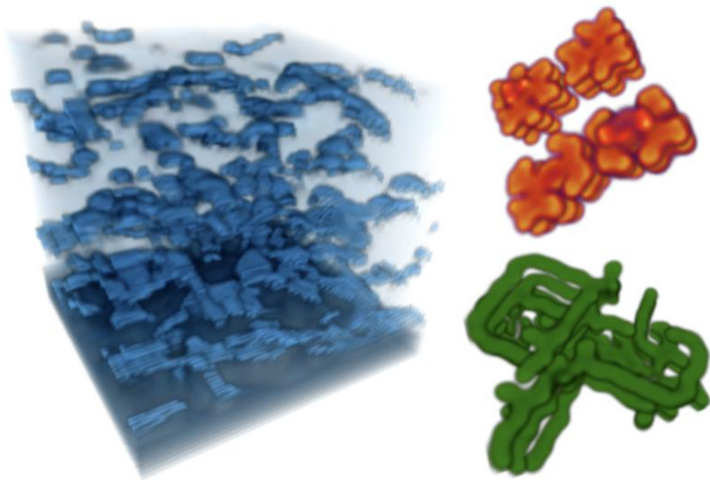
We now can use the GPU like a data-parallel programming system.

Fragment shader function is mapped over 512 x 512 element collection.

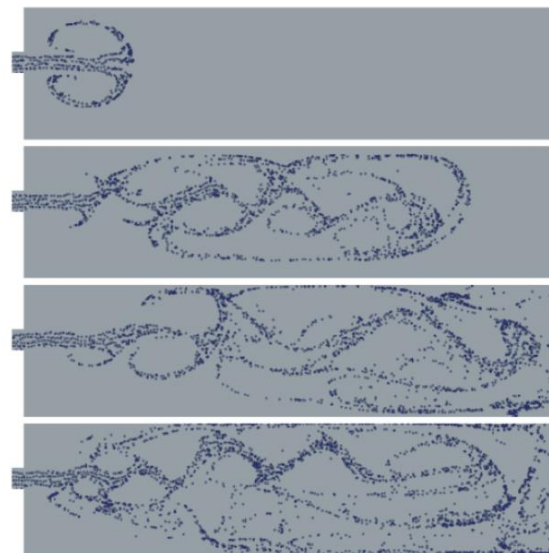


GPUs in 2001~2003

- General Purpose computation on GPUs



Coupled Map Lattice Simulation [Harris 02]



Sparse Matrix Solvers [Bolz 03]



Ray Tracing on Programmable Graphics Hardware [Purcell 02]

GPU computation

Parallelism

- Task Parallelism
 - Distributes tasks across multiple cores
 - CPU works better than GPU
- Data Parallelism
 - Distributes data across multiple cores
 - Suitable to GPU architecture

Computer Architecture

- A form of computation in which many calculations are carried out simultaneously
- Solve sub-problems of a problem concurrently

Single core processor

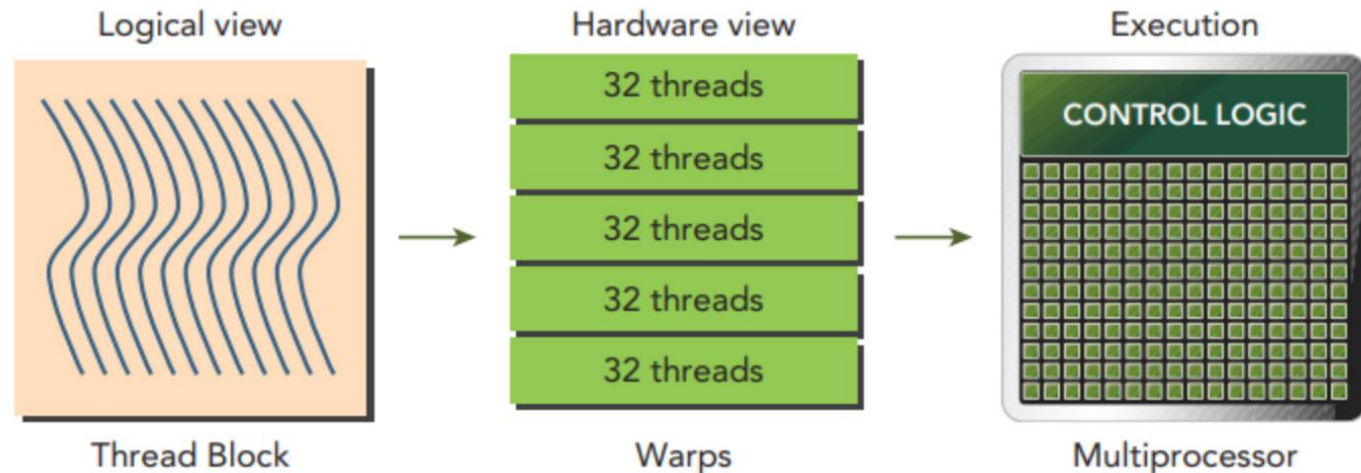
Vector processor

SISD Single instruction stream Single data stream	SIMD Single instruction stream Multiple data stream
MISD Multiple instruction stream Single data stream	MIMD Multiple instruction stream Multiple data stream

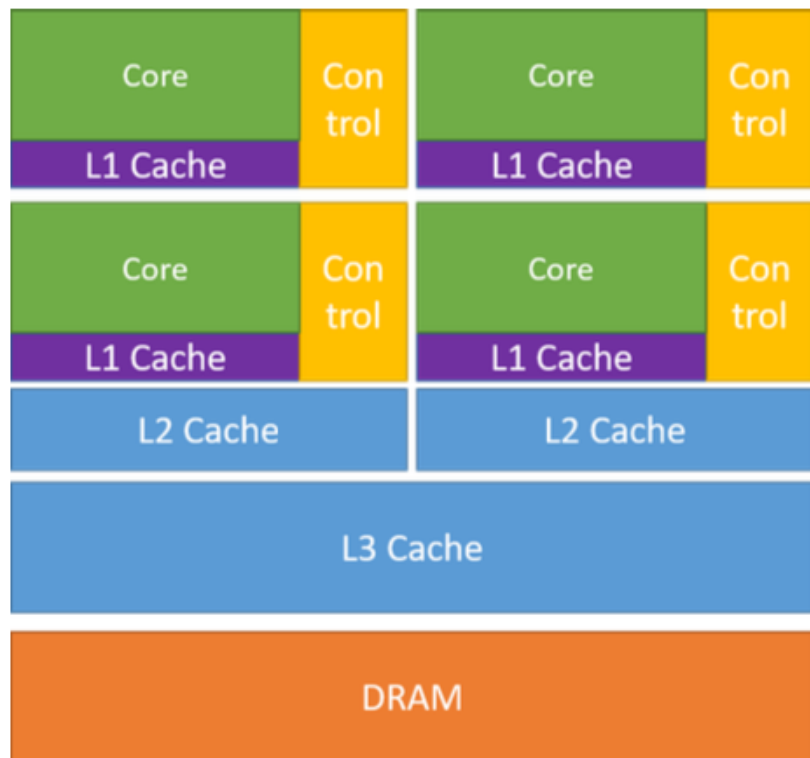
Not covered

Multi-core processor

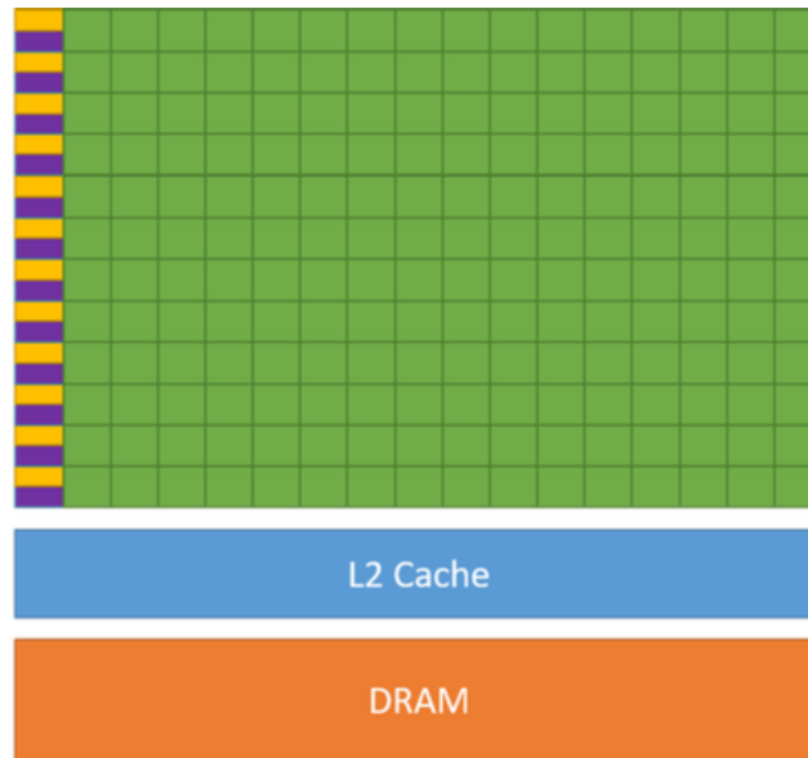
- The architecture of GPU is called SIMT
- **S**ingle **I**nstruction, **M**ultiple **T**hreads (SIMT)
 - A group of threads is controlled by a control unit
(모든 스레드는 동일한 명령어를 실행하고, 각 스레드는 할당된 private data에 대해 작업을 수행)



CPU vs. GPU



**Multi-core
CPU**



GPU

CPU vs. GPU

	CPU	GPU
Strength	• High performance processing core • Efficient irregular workflow handling • Branch prediction • Efficient handling for random memory access pattern • Well-organized cache hierarchy • Large memory space	• A massive number of cores • But, less powerful than CPU core • Much higher performance than CPU • In a perspective of FLOPS FLOPS(FLOating point Operations Per Second)는 컴퓨터가 1초 동안 수행할 수 있는 부동소수점 연산 횟수 (FLOPs = 클럭속도 × 코어개수 × 사이클당 연산 수)
Weakness	• A small number of cores (up to 64) • More space for controls • Lower performance than GPU • In a perspective of FLOPS	• Small memory space • High bandwidth memory = expensive • Performance penalty for irregular workflow • Weak for random memory access pattern

CPU vs. GPU

	Nvidia RTX 3080	Raden RX 6800	Intel i9 11900K	AMD Ryzen 9 5950X	Intel Xeon Gold 6254
# of cores	8704	3840	8	16	18
base clock	1.44 Ghz	1.82 Ghz	3.50 Ghz	3.4 Ghz	3.10 Ghz
boost clock	1.71 Ghz	2.11 Ghz	5.30 Ghz	4.9 Ghz	4.00 Ghz
Memory type	GDDR6X	GDDR6	DDR4-3200	DDR4-3200	DDR4-2933
Memory size	10GB	16 GB	~128GB	~128GB	~ 1TB
L2 Cache size	5 MB	4 MB	4 MB	8 MB	18 MB
L3 Cache size	-		16 MB	64 MB	24.75 MB

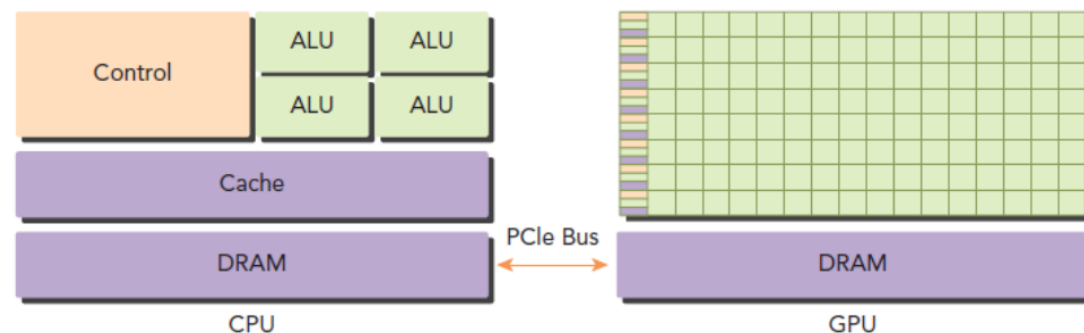
Heterogeneous Architecture

- A heterogeneous architecture consisting of more than one type of computing resources
- Examples
 - A desktop PC having both multi-core CPUs and GPUs
 - A multi-GPU system consisting of different types of GPUs



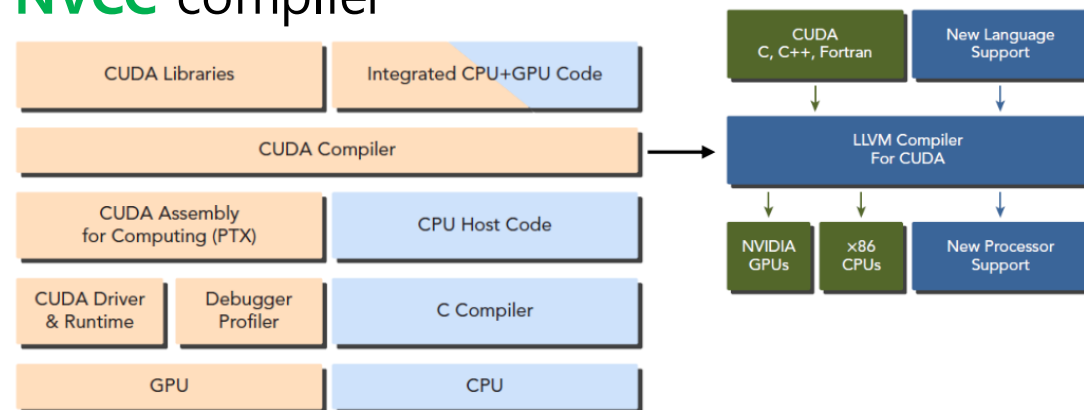
Heterogeneous Architecture

- Host (= CPU)
 - Host code, host memory, ...
- Device (= GPU)
 - Device code, device memory, ...
- Host code + Device code
 - The host code runs on CPU
 - The device code runs on GPU

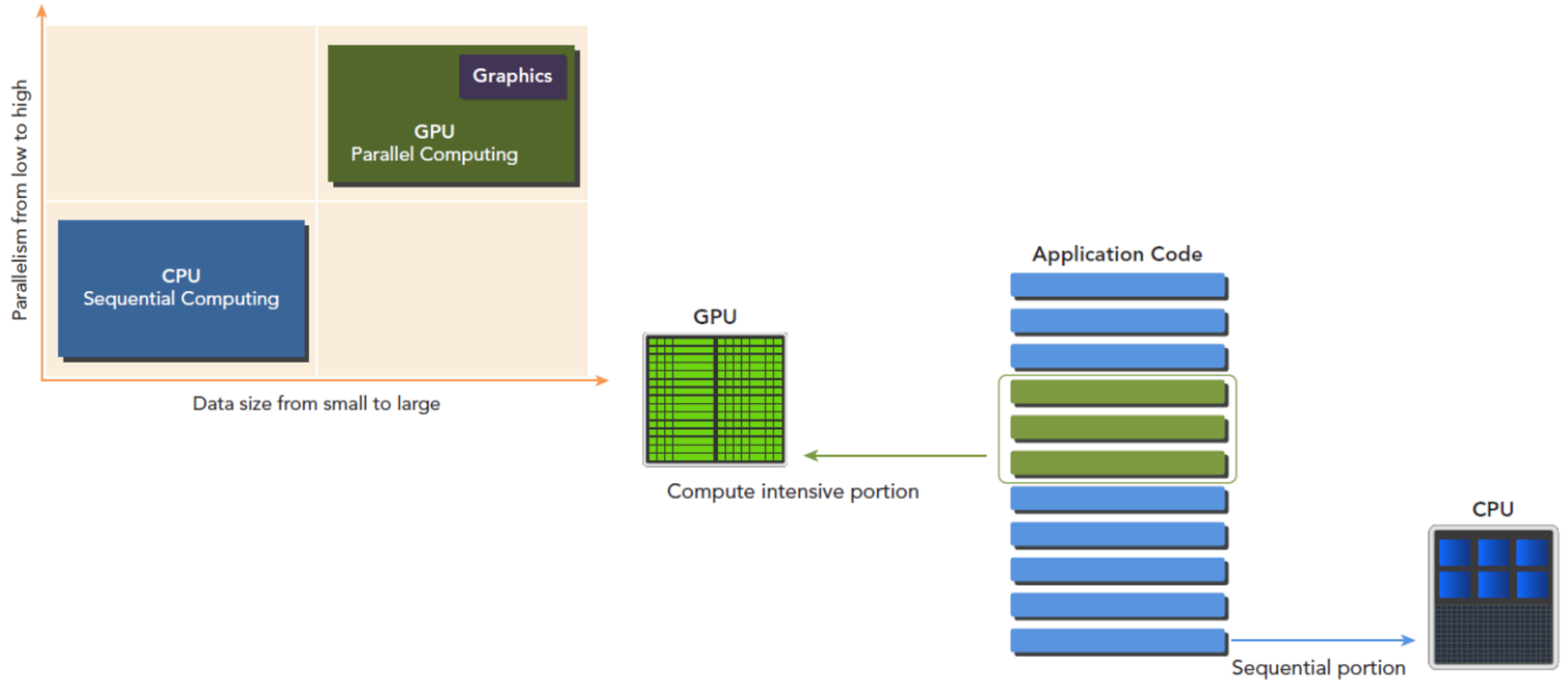


* arithmetic and logical unit, ALU

NVCC compiler

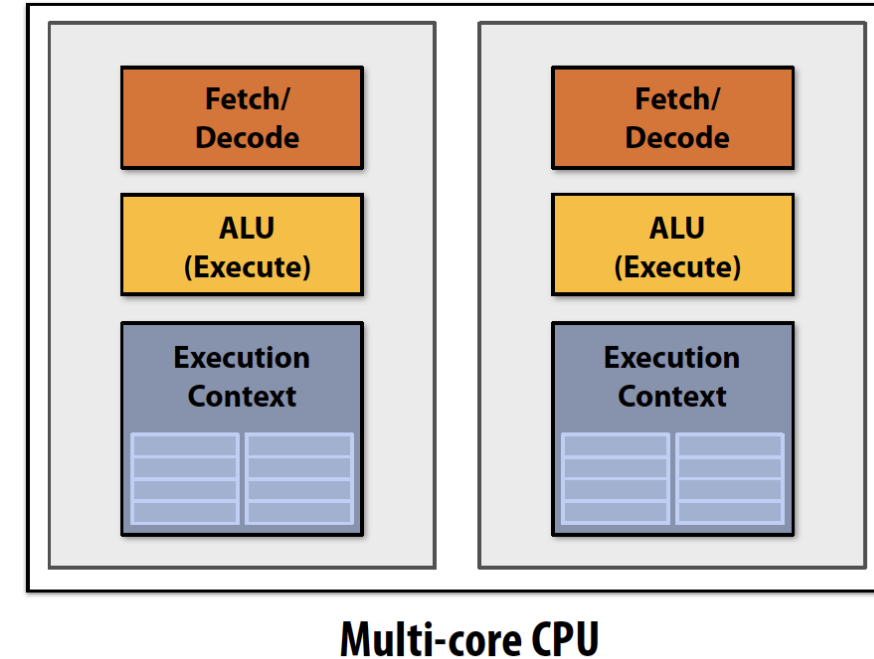


Heterogeneous Architecture



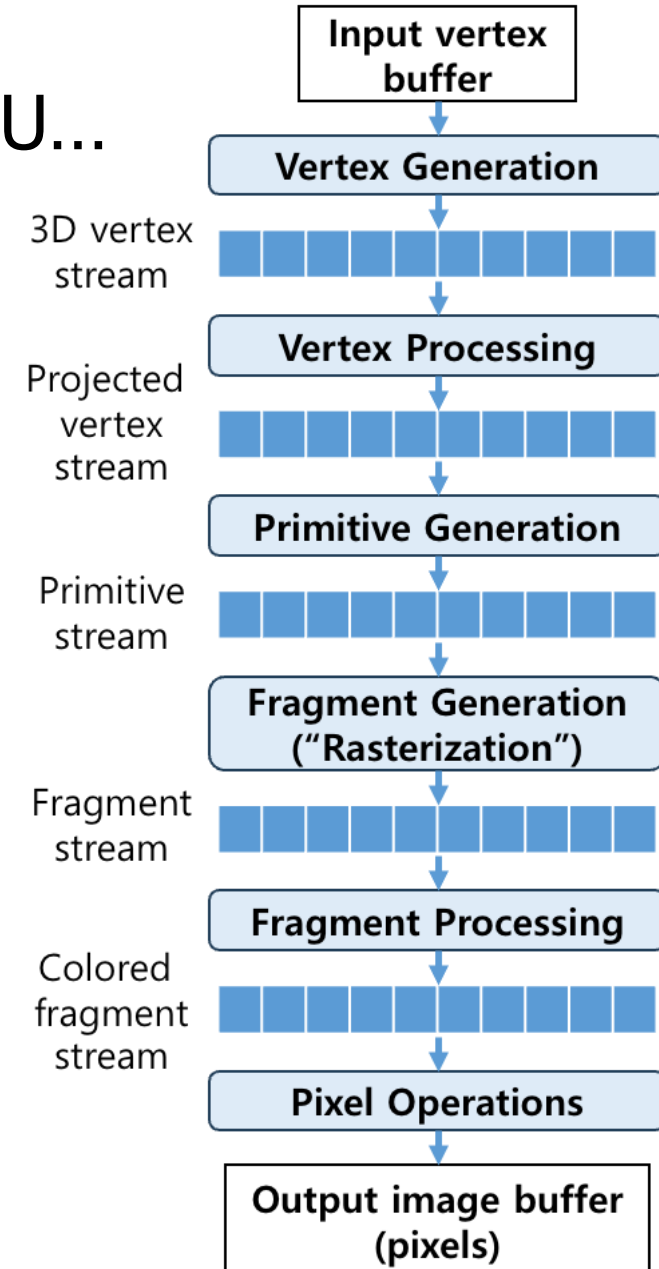
Review: how to run code on a CPU

- OS loads program text into memory
- OS selects CPU execution context
- OS interrupts processor, prepares execution context (sets contents of registers, program counter, etc. to prepare execution context)
- Go!
- Processor begins executing instructions from within the environment maintained in the execution context.



Review: how to run code on a GPU (prior to 2007)

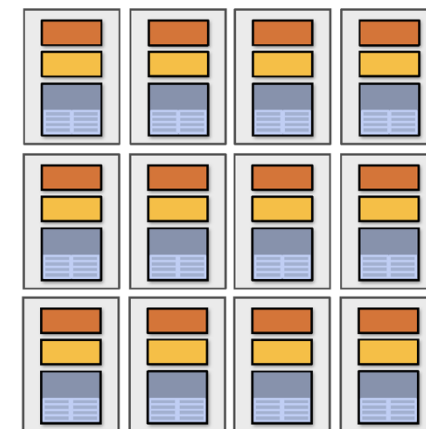
- Lets say a user wants to draw a picture using a GPU...
 - Application (via graphics driver) provides GPU vertex and fragment shader program binaries
 - Application sets graphics pipeline parameters (e.g., output image size)
 - Application provides hardware a buffer of vertices
 - Go! (`drawPrimitives(vertex_buffer)`)



NVIDIA Tesla Architecture (after 2007)

- Lets say a user wants to draw a picture using a GPU...
- (GeForce 8xxx series GPUs) First alternative, non-graphics-specific ("compute mode") interface to GPU hardware
- A user wants to run a non-graphics program on the GPU's programmable cores...
 - Application can allocate buffers in GPU memory and copy data to/from buffers
 - Application (via graphics driver) provides GPU a single kernel program binary
 - Application tells GPU to run the kernel in an SPMD fashion ("run N instances")
 - Go! (launch(myKernel, N))

this is a far simpler operation than drawPrimitives()



CUDA programming Language

- Introduced in 2007 with NVIDIA Tesla architecture
- “C-like” language to express programs that run on GPUs using the compute-mode hardware interface
- Relatively low-level: CUDA’s abstractions closely match the capabilities/performance characteristics of modern GPUs (design goal: maintain low abstraction distance)
- Note: OpenCL is an open standards version of CUDA
 - CUDA only runs on NVIDIA GPUs
 - OpenCL runs on CPUs and GPUs from many vendors
 - Almost everything I say about CUDA also holds for OpenCL
 - CUDA is better documented, thus I find it preferable to teach with

CUDA programming Language

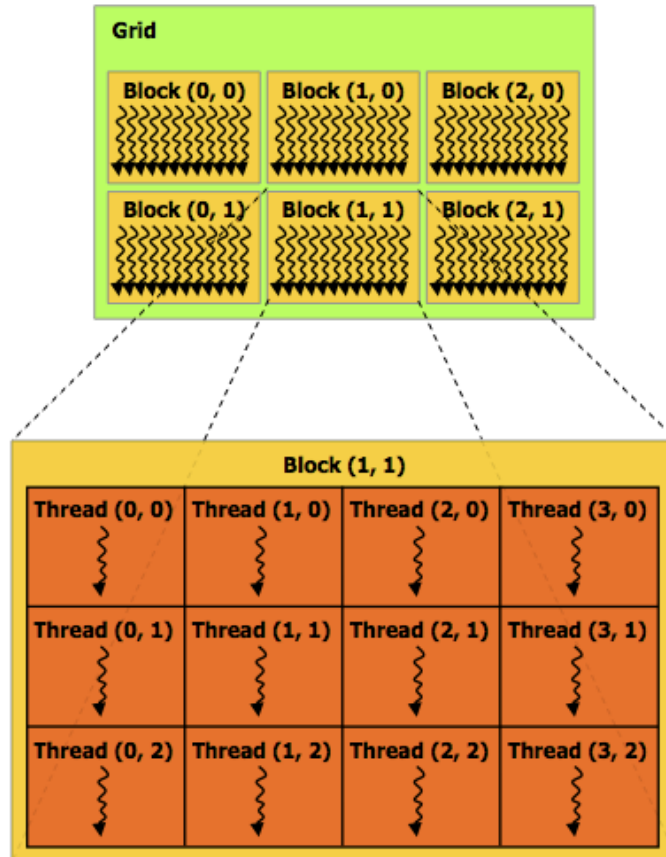
- Specifically, be careful with the use of the term CUDA thread.

A CUDA thread presents a similar abstraction as a pthread in that both correspond to logical threads of control, but the implement of a CUDA thread is very different.

CUDA programs consist of a hierarchy of concurrent threads

- Thread IDs can be up to 3D (2D example below) Multi-dimensional thread ids are convenient for problems that are naturally N-D

From slides of NVIDIA Corporation



```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x,
               Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>>(A, B, C);
```

Regular application thread running on CPU (the “host”)

Hello CUDA

```
!nvcc --version
```

```
!pip install git+https://github.com/andreinechaev/nvcc4jupyter.git
```

```
%load_ext nvcc4jupyter
```

```
%%cuda
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>

__global__ void helloCUDA(void)
{
    printf("Hello CUDA from GPU!(Block:%d, Thread:%d)\n",
           blockIdx.x, threadIdx.x);
}

int main(void)
{
    printf("Hello GPU from CPU!\n");
    helloCUDA <<<1,10>>>();
    cudaDeviceSynchronize(); // stream 동기화
    return 0;
}
```

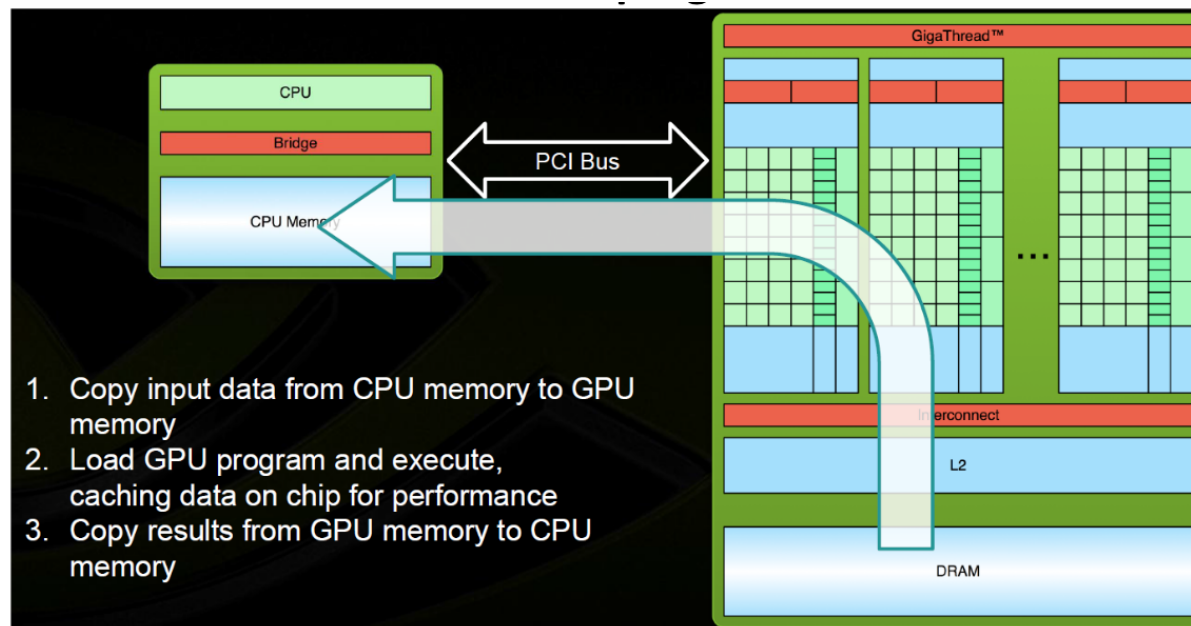
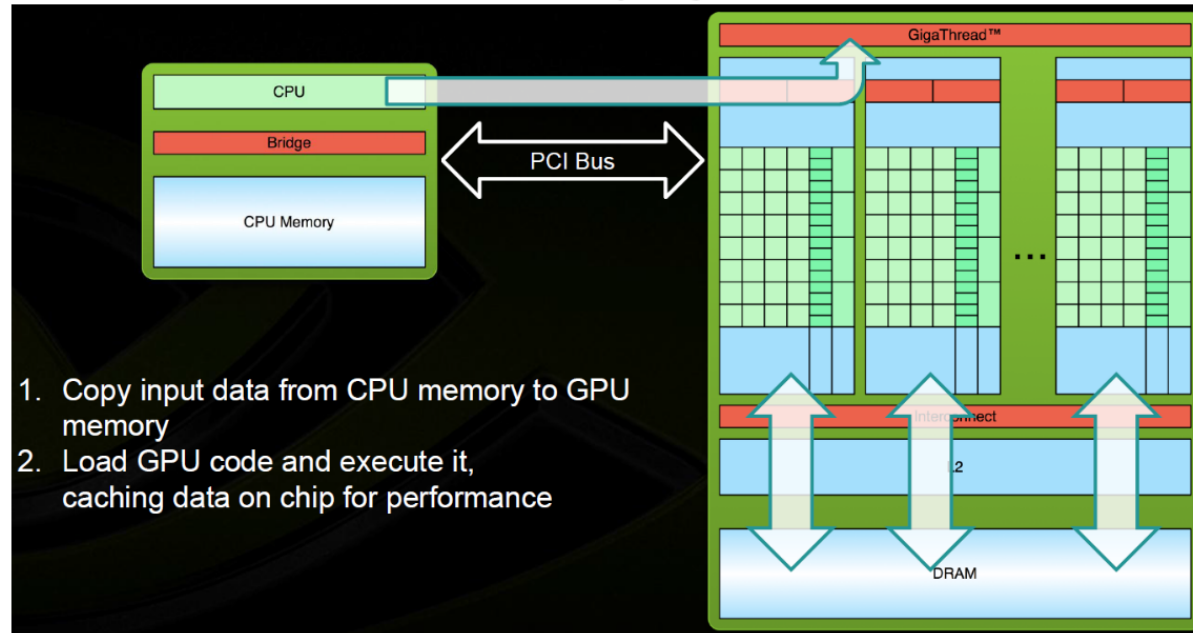
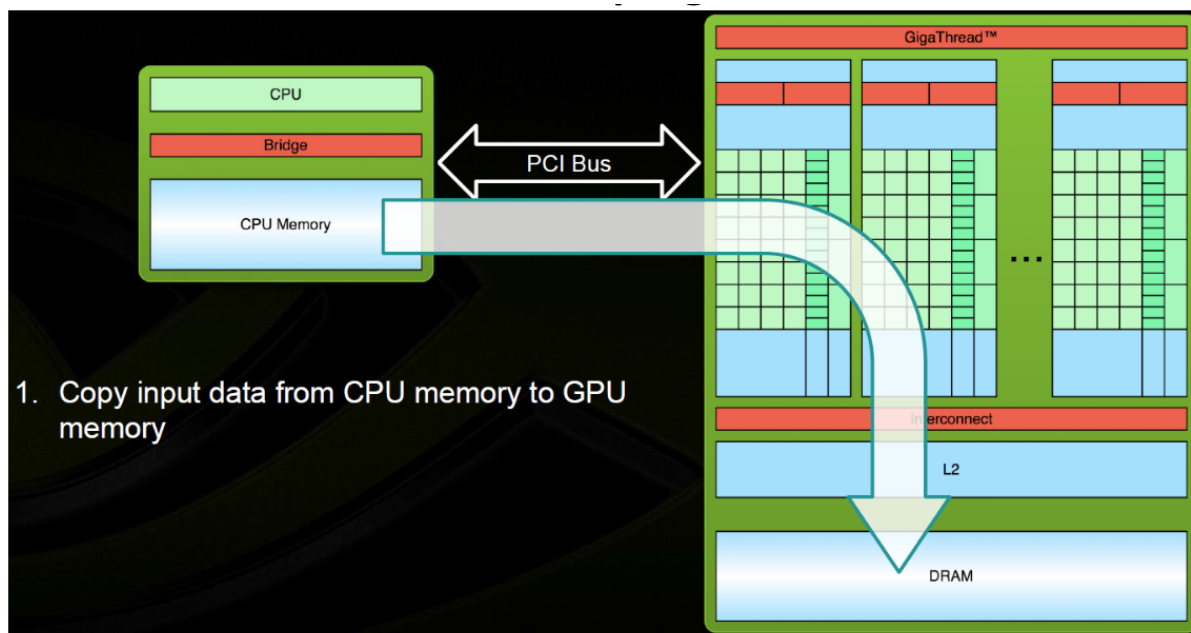
#CUDA를 위한 compiler
#• Host code와 device code를 분리해서 compile
#• Device code → nvcc,
#• Host code → host compiler

```
// add 함수를 1번 실행
function<<<1,1>>>(parameters);

// N개의 블록으로 1개의 스레드를 통해 실행(Parallel Block)
function<<<N,1>>>(parameters);

// 1개의 블록으로 N개의 스레드를 통해 실행(Parallel Thread)
function<<<1,N>>>(parameters);
```

General workflow of a CUDA Program



Basic API for Device Memory

STANDARD C FUNCTIONS	CUDA C FUNCTIONS
<code>malloc</code>	<code>cudaMalloc</code>
<code>memcpy</code>	<code>cudaMemcpy</code>
<code>memset</code>	<code>cudaMemset</code>
<code>free</code>	<code>cudaFree</code>

Data copy between host and device

- `cudaMemcpy`(void* dst, const void* src, size_t size, enum cudaMemcpyKind kind)
- `cudaMemcpyHostToHost`
- `cudaMemcpyHostToDevice`
- `cudaMemcpyDeviceToHost`
- `cudaMemcpyDeviceToDevice`
- `cudaMemcpyDefault`

Vector sum with CUDA

```
int main(void) {
    int *a, *b, *c, *h_c; // Vectors on the host
    int *d_a, *d_b, *d_c; // Vectors on the device
    int memSize = sizeof(int)*NUM_DATA;

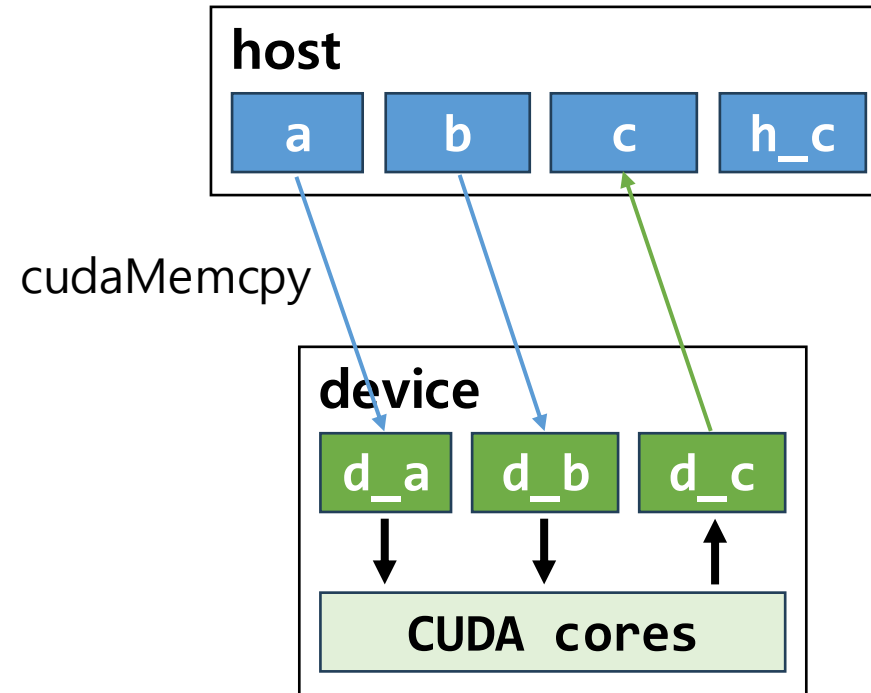
    a = new int[NUM_DATA];
    b = new int[NUM_DATA];
    c = new int[NUM_DATA];
    h_c = new int[NUM_DATA];
    ...

    // memory allocation in device
    cudaMalloc(&d_a, memSize);
    cudaMalloc(&d_b, memSize);
    cudaMalloc(&d_c, memSize);
    ...

    cudaMemcpy(d_a, a, memSize, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, memSize, cudaMemcpyHostToDevice);
    ...

    vecAdd<<<1, NUM_DATA >>>(d_a, d_b, d_c);
    ...

    cudaMemcpy(c, d_c, memSize, cudaMemcpyDeviceToHost);
}
```



Performance profiling

- Data Transfer time from host to device
- Computation time
- Data Transfer time from device to host

Performance Result!

- 왜 안빠름?
- DATA_SIZE 늘리며 성능 비교 해보기

```
VectorSum on Host: 0.001521 ms
```

```
Data Trans. : Host -> Device: 0.033843 ms
```

```
Computation: 37.4369 ms
```

```
Data Trans. : Device -> Host: 0.026216 ms
```

```
Cuda Total: 37.5472 ms
```